

Projeto DimDim

Containers em Nuvem

1º Checkpoint · 2º Semestre · DevOps Tools & Cloud Computing
Azure Container Registry e Azure Container Instances

ALUNA	Natalia Cristina de Souza — RM 564099
GRUPO	_____
INTEGRANTES	_____
PROFESSOR	João Carlos Menk
REPOSITÓRIO	_____
VÍDEO	_____
ASSINATURA	Azure for Students · região East US 2

01 O que foi solicitado, e onde está

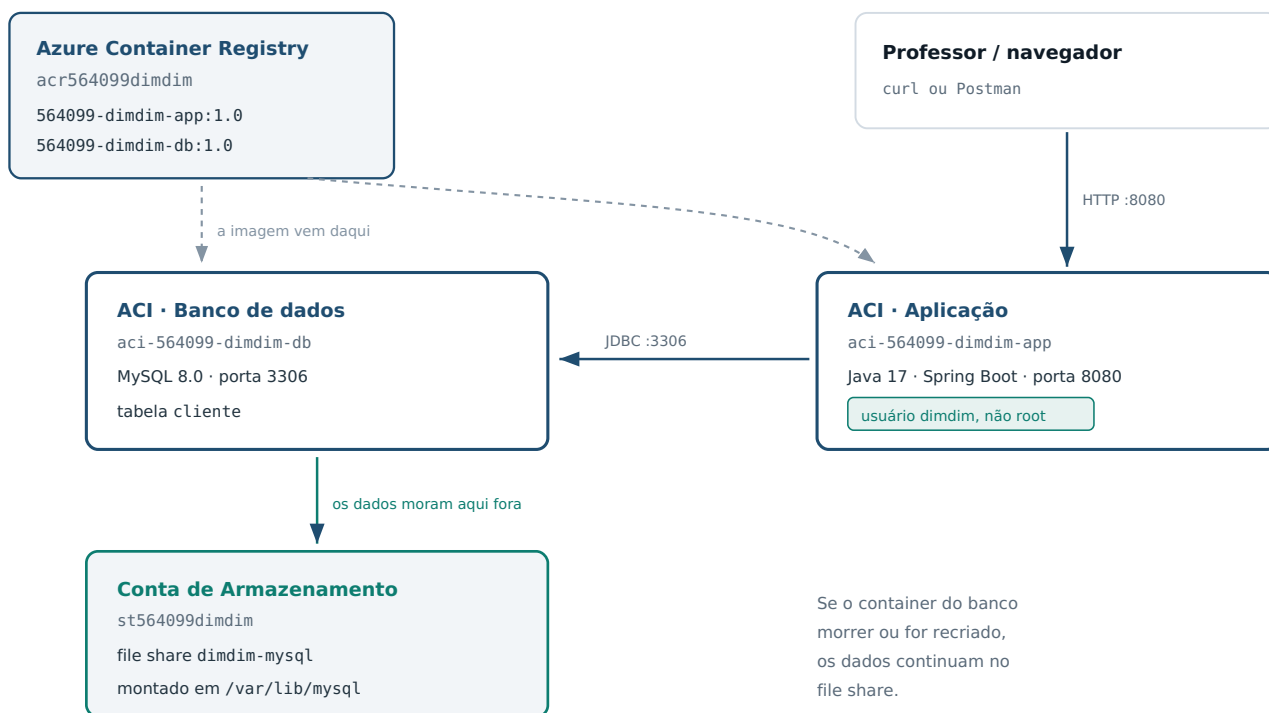
Cada exigência do enunciado, com o arquivo ou a evidência que a atende.

EXIGÊNCIA DO PROFESSOR	ONDE ESTÁ	SITUAÇÃO
App Java ou .NET containerizado	app/Dockerfile	OK
Banco relacional containerizado (H2 proibido)	db/Dockerfile – MySQL 8.0	OK
Dockerfile do banco e do app	app/ e db/	OK
Build e teste local antes da nuvem	docker-compose.yml	OK
Imagens no ACR com RM como prefixo	564099-dimdim-app · 564099-dimdim-db	OK
Dois ACIs com RM como prefixo	aci-564099-dimdim-app · -db	OK
Persistência em Conta de Armazenamento	st564099dimdim / dimdim-mysql	OK
Evidência de CRUD por SELECT	seção 05 deste documento	OK
Recursos criados via Azure CLI	scripts/01 ... 06	OK
README com How To (build e push)	README.md	OK
App sem privilégio de root	USER dimdim	OK
DDL das tabelas no repositório	db/init/01-ddl.sql	OK
JSONs de teste dos 4 verbos	json/	OK

Sem credenciais no código	variáveis de ambiente seguras	OK
Vídeo 720p com explicação falada	roteiro na seção 07	a gravar

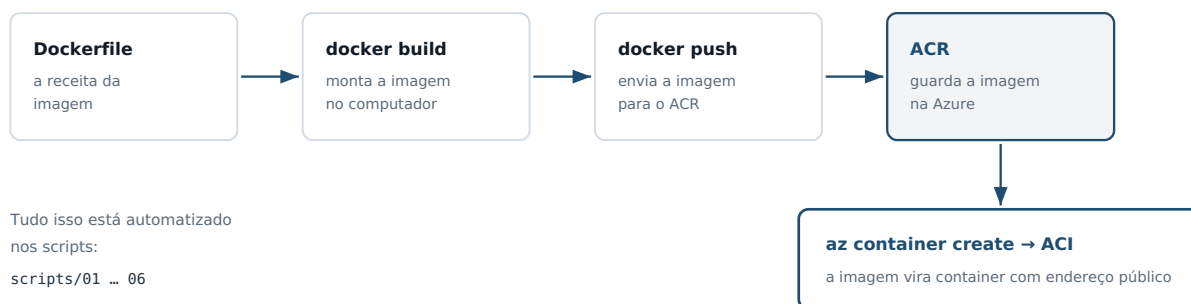
02 A arquitetura

Duas imagens guardadas no registry, dois containers rodando na nuvem, e um disco externo onde os dados do banco realmente moram.



03 Como a imagem chega até a nuvem

O caminho percorrido por cada uma das duas imagens, do código-fonte até o container rodando com endereço público.



04 Os recursos, na Azure

```
$ az resource list --resource-group rg-564099-dimdim --output table
```

Recurso	Tipo	Regiao
-----	-----	-----
acr564099dimdim	Microsoft.ContainerRegistry/registries	eastus2
st564099dimdim	Microsoft.Storage/storageAccounts	eastus2
aci-564099-dimdim-db	Microsoft.ContainerInstance/containerGroups	eastus2
aci-564099-dimdim-app	Microsoft.ContainerInstance/containerGroups	eastus2

```
$ az acr repository list --name acr564099dimdim --output table
```

```
Result
-----
564099-dimdim-app
564099-dimdim-db
```

As duas imagens levam o RM 564099 como prefixo, conforme exigido.

```
$ az container list --resource-group rg-564099-dimdim --output table
```

Container	Endereco
-----	-----
aci-564099-dimdim-app	564099-dimdim-app.eastus2.azurecontainer.io
aci-564099-dimdim-db	564099-dimdim-db.eastus2.azurecontainer.io

Por que East US 2 e não Brasil? A assinatura Azure for Students aplica uma política que só libera cinco regiões: canadacentral, southcentralus, chilecentral, eastus2 e northcentralus. Tentar criar em brazilsouth devolve RequestDisallowedByAzure.

05 Evidências do CRUD

Cada operação da API seguida do SELECT direto no banco. É o SELECT que comprova a persistência — e é o que vale 30 pontos no enunciado.

Estado inicial

```
$ mysql -h 564099-dimdim-db.eastus2.azurecontainer.io -u dimdim -p -D dimdim -e "SELECT * FROM cliente;"
```

id_cliente	nm_cliente	nr_cpf	ds_email	vl_saldo
1	Steves Jobs	11122233344	steves@dimdim.com.br	15000.00
2	Maria Fernandes	55566677788	maria@dimdim.com.br	2350.75

CREATE — POST /clientes

```
$ curl -X POST ../clientes -H "Content-Type: application/json" -d @json/cliente-post.json
```

```
{
  "id": 3,
  "nome": "Joao Carlos Menk",
  "cpf": "99988877766",
  "email": "joao.menk@dimdim.com.br",
  "saldo": 8750.5
}
```

```
SELECT * FROM cliente;
```

id_cliente	nm_cliente	nr_cpf	ds_email	vl_saldo
1	Steves Jobs	11122233344	steves@dimdim.com.br	15000.00
2	Maria Fernandes	55566677788	maria@dimdim.com.br	2350.75
3	Joao Carlos Menk	99988877766	joao.menk@dimdim.com.br	8750.50

O registro de id 3 foi gravado no banco.

READ — GET /clientes/3

```
$ curl ../clientes/3
```

```
{
  "id": 3,
  "nome": "Joao Carlos Menk",
  "cpf": "99988877766",
  "email": "joao.menk@dimdim.com.br",
  "saldo": 8750.5
}
```

UPDATE — PUT /clientes/3

```
$ curl -X PUT ../clientes/3 -H "Content-Type: application/json" -d @json/cliente-put.json
```

```
{
  "id": 3,
  "nome": "Joao Carlos Menk",
  "cpf": "99988877766",
  "email": "joao.menk@fiap.com.br",
  "saldo": 12300.0
}
```

```
SELECT * FROM cliente;
```

id_cliente	nm_cliente	nr_cpf	ds_email	vl_saldo
1	Steves Jobs	11122233344	steves@dimdim.com.br	15000.00
2	Maria Fernandes	55566677788	maria@dimdim.com.br	2350.75
3	Joao Carlos Menk	99988877766	joao.menk@fiap.com.br	12300.00

No banco, o e-mail passou de @dimdim.com.br para @fiap.com.br e o saldo de 8.750,50 para 12.300,00.

DELETE — DELETE /clientes/3

```
$ curl -X DELETE ../clientes/3
```

resposta: HTTP 204

```
SELECT * FROM cliente;
```

id_cliente	nm_cliente	nr_cpf	ds_email	vl_saldo
1	Steves Jobs	11122233344	steves@dimdim.com.br	15000.00
2	Maria Fernandes	55566677788	maria@dimdim.com.br	2350.75

O registro de id 3 desapareceu da tabela.

06 Segurança e persistência

O container da aplicação não roda como root

```
$ az container exec -g rg-564099-dimdim -n aci-564099-dimdim-app --exec-command "id"
```

```
uid=100(dimdim) gid=101(dimdim) groups=101(dimdim)
```

No Dockerfile: RUN addgroup -S dimdim && adduser -S dimdim -G dimdim seguido de USER dimdim.

Os dados sobrevivem ao container

Para provar a persistência de verdade, foi inserido um registro que **não existe no script de DDL** — se ele voltasse do DDL, a prova não valeria nada. Em seguida o container do banco foi reiniciado.

```
$ az container restart -g rg-564099-dimdim -n aci-564099-dimdim-db
```

```
SELECT * FROM cliente; (depois do restart)
```

id_cliente	nm_cliente	nr_cpf	ds_email	vl_saldo
1	Steves Jobs	11122233344	steves@dimdim.com.br	15000.00
2	Maria Fernandes	55566677788	maria@dimdim.com.br	2350.75
4	Teste Persistencia	12312312312	persistencia@dimdim.com.br	999.99

O registro de id 4 sobreviveu, e o id 3 — apagado antes — continuou ausente. O file share preservou o estado exato da tabela.

Nenhuma credencial no código

- Senhas entram nos containers como `--secure-environment-variables`, que não aparecem no portal nem em `az container show`.
- O `application.properties` lê tudo de variável de ambiente: `${DB_HOST}`, `${DB_USER}`, `${DB_PASSWORD}`.
- O arquivo `.env` local está no `.gitignore` e nunca foi versionado.

07 Roteiro do vídeo

O script `gravar-video.sh` conduz a gravação: ele mostra a fala na tela, você lê em voz alta, aperta ENTER, e o comando roda sozinho. São dez passos.

Antes de gravar: aumente a fonte do terminal, feche abas pessoais, deixe o portal da Azure aberto numa aba, confira o microfone e grave em 720p ou mais.

Passo 1 · Abertura

FALE ISSO

Oi professor. Sou a Natalia, RM 564099. Esse é o checkpoint de containers em nuvem, o projeto DimDim. Vou começar mostrando os recursos que eu criei na Azure.

Passo 2 · Os containers

FALE ISSO

São quatro recursos: o Container Registry, a conta de armazenamento, e as duas instâncias de container, uma do banco e uma da aplicação.

Passo 3 · As imagens no ACR

FALE ISSO

Aqui estão as duas imagens que eu registrei no Container Registry. Repare que as duas começam com o meu RM, como o enunciado pediu.

Passo 4 · Sem root

FALE ISSO

O container da aplicação não roda como root. Ele roda com um usuário que eu criei no Dockerfile, chamado `dimdim`.

Passo 5 · Estado inicial

FALE ISSO

Agora o CRUD. Primeiro o estado inicial: dois clientes na tabela.

Passo 6 · CREATE

FALE ISSO

Agora eu vou inserir um cliente novo. Isso é o CREATE.

Passo 7 · READ

FALE ISSO

O cliente novo apareceu no banco. Agora o READ: buscando esse cliente pela API.

Passo 8 · UPDATE

FALE ISSO

Agora o UPDATE. Vou mudar o e-mail e o saldo desse cliente.

Passo 9 · DELETE

FALE ISSO

O e-mail e o saldo mudaram no banco. Agora o DELETE, removendo esse cliente.

Passo 10 · Fechamento

FALE ISSO

O registro sumiu da tabela. Os quatro comandos do CRUD funcionaram, e cada um foi confirmado com um SELECT direto no banco. Todos os recursos foram criados por linha de comando, e os scripts estão no GitHub junto com os Dockerfiles e o DDL das tabelas. Obrigada, professor.

08 Se o professor perguntar

Por que o app não pode rodar como root?

Se alguém invadir o container, com root faz o que quiser lá dentro. Criei um usuário `dimdim` no Dockerfile e usei o comando `USER`, então o processo roda sem privilégio administrativo.

E se o container do banco morrer, perde tudo?

Não. Os dados ficam num file share da conta de armazenamento, montado em `/var/lib/mysql`. Eu testei: reiniciei o container e os dados continuaram lá.

Por que MySQL e não H2?

O H2 é banco em memória e o enunciado não permite. Além disso ele não serviria aqui, porque o banco precisa ser um container separado, com persistência própria.

O que é multi-stage no Dockerfile?

São duas etapas. Na primeira eu uso a imagem do Maven para compilar o Java. Na segunda copio só o `.jar` pronto para uma imagem de JRE, bem menor. A imagem final não carrega o Maven nem o código-fonte.

Como o app sabe o endereço do banco?

Por variável de ambiente. Quando crio o ACI da aplicação, passo o endereço do banco e a senha. Nada disso está escrito no código.

Onde estão as senhas?

Em lugar nenhum do código ou do GitHub. Entram como `--secure-environment-variables`, que nem aparecem no portal da Azure.

09 Antes de entregar

- ☐ Vídeo em 720p, com voz, começando pelos recursos na Azure
- ☐ Os quatro verbos do CRUD, cada um seguido de SELECT no banco
- ☐ Código no GitHub, com acesso liberado ao professor
- ☐ Link do vídeo acessível (sem acesso = nota zero)
- ☐ PDF <nome_grupo>_container.pdf com folha de rosto, RMs e os dois links
- ☐ Upload no Teams feito pelo representante do grupo

Depois que a nota sair: apague os recursos para não consumir o crédito da assinatura — `az group delete --name rg-564099-dimdim --yes`. Antes disso não, porque o professor pode querer ver os recursos no ar.